

## Pemrograman Dasar

### *Pseudocode dan Penjelasan Logika Problem 1: The Last Astronaut*

PROGRAM the\_last\_astronaut

DECLARE

n, k  
alive[1..n]  
current  
eliminated  
counted  
candidate  
aliveCount

BEGIN

INPUT n  
INPUT k

FOR i FROM 1 UNTIL n  
    alive[i] = true  
END FOR

aliveCount = n  
current = 1

WHILE aliveCount > 1  
    candidate = current  
    counted = 1

    WHILE counted < k  
        candidate = NEXT\_ALIVE(candidate)  
        counted = counted + 1  
    END WHILE

    eliminated = candidate  
    DISPLAY "Astronot dieliminasi: ", eliminated

    alive[eliminated] = false  
    aliveCount = aliveCount - 1

    IF aliveCount > 0  
        current = NEXT\_ALIVE(eliminated)  
    END IF

    IF (eliminated MOD 2 == 0)

```

        k = k + 2
    ELSE
        k = k - 1
    END IF
    IF k < 2
        k = 2
    END IF
END WHILE

DISPLAY "Astronot terakhir yang bertahan: ", current
END

FUNCTION NEXT_ALIVE(from)
    next = from + 1
    IF next > n
        next = 1
    END IF
    WHILE alive[next] == false
        next = next + 1
        IF next > n
            next = 1
        END IF
    END WHILE
    RETURN next
END FUNCTION

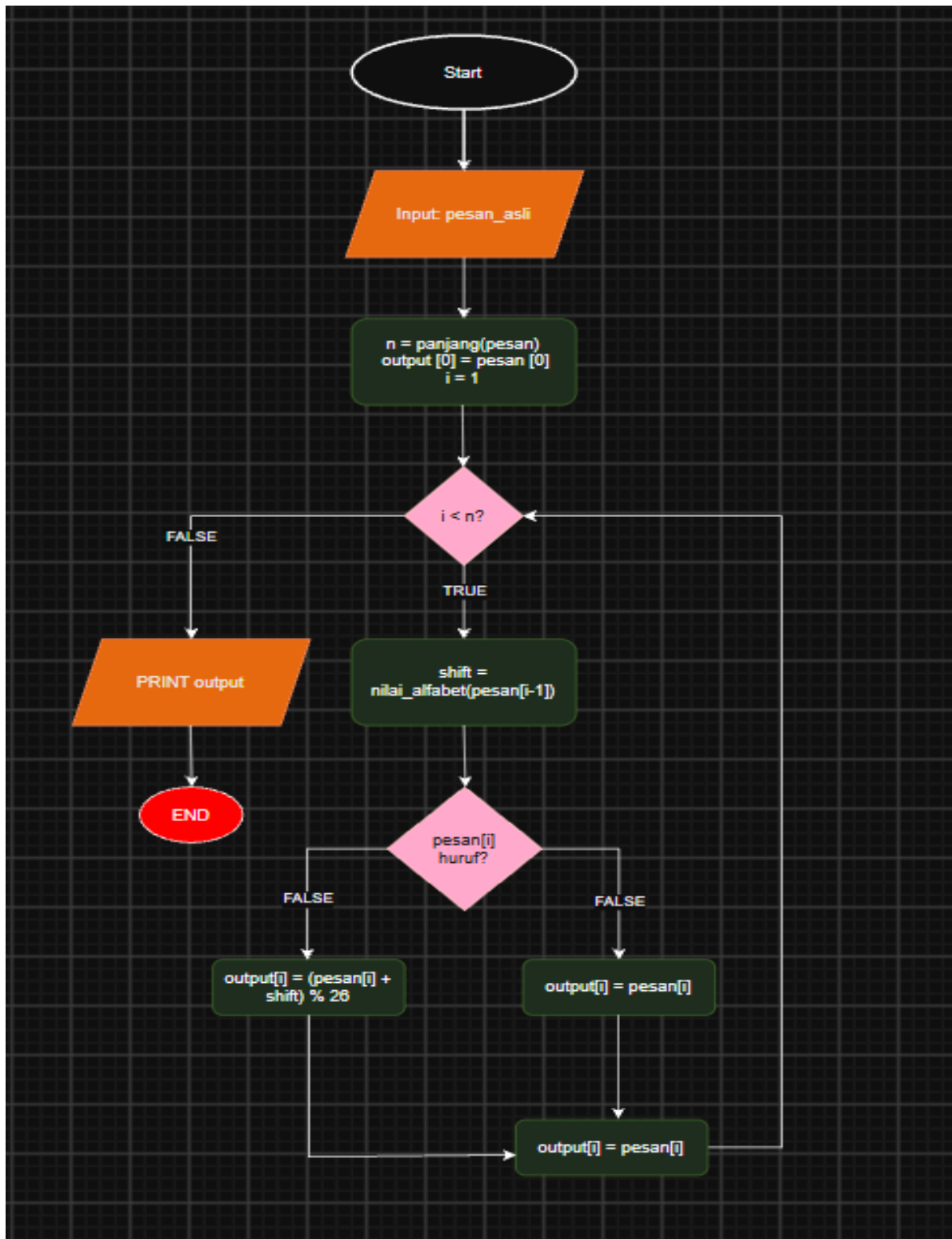
```

### Penjelasan

Masalah ini merupakan variasi dari Josephus Problem dengan nilai K yang berubah-ubah. Astronot direpresentasikan sebagai array boolean `alive[1..n]` yang menandai, apakah astronot pada nomor tersebut masih berada dalam lingkaran. Karena posisi astronot yang sudah dieliminasi tidak boleh ikut dihitung lagi, sistem lingkaran disimulasikan menggunakan fungsi bantu `NEXT_ALIVE(from)` yang mencari astronot hidup terdekat setelah posisi tertentu, dan otomatis lompat kembali ke nomor 1 ketika mencapai ujung array (perilaku melingkar / circular).

Pada setiap putaran, perhitungan dimulai dari posisi `current` yang dianggap sebagai hitungan pertama, terus berpindah ke astronot hidup berikutnya hingga total hitungan mencapai K, sehingga astronot pada hitungan ke-K itulah yang dieliminasi. Setelah eliminasi, posisi awal untuk putaran berikutnya diperbarui menjadi astronot setelah yang sebelumnya dieliminasi, dan nilai K disesuaikan berdasarkan paritas nomor yang dieliminasi (genap menambah K sebesar 2, ganjil mengurangi K sebesar 1), dengan batas bawah K dijaga tidak boleh kurang dari 2. Proses ini berulang hingga hanya tersisa satu astronot, yang kemudian menjadi output akhir program.

## Flowchart dan Penjelasan Logika Problem 2: Alien-In-The-Middle



### Penjelasan

Program enkripsi *Alien-In-The-Middle* diawali dengan menerima masukan berupa teks dari pengguna dan menghitung panjang string secara manual melalui iterasi penanda akhir *null-terminator* ( $\backslash 0$ ). Setelah panjang pesan diketahui, program menentukan karakter pertama pada indeks  $i = 0$  langsung ke dalam string keluaran tanpa mengalami perubahan apa pun.

Karakter pertama ini berfungsi sebagai jangkar utama karena proses enkripsi untuk karakter-karakter selanjutnya akan bergantung pada nilai alfabet dari karakter yang tepat berada di posisi sebelumnya.

Proses enkripsi berlanjut melalui perulangan dari indeks kedua ( $i = 1$ ) hingga akhir string. Pada setiap iterasi, program mengevaluasi karakter saat ini (*current*) dan karakter sebelumnya (*previous*). Nilai pergeseran (*shift*) diperoleh dari konversi posisi alfabet karakter sebelumnya ke dalam rentang numerik 1 hingga 26 ( $A/a = 1$  sampai  $Z/z = 26$ ). Jika karakter saat ini merupakan huruf alfabet, posisinya digeser sejauh nilai *shift* menggunakan rumus aritmatika modulo 26 untuk memastikan pergeseran kembali melingkar ke awal alfabet jika melebihi batas. Proses ini dilakukan dengan tetap mempertahankan tipe huruf aslinya (huruf kapital tetap kapital, dan huruf kecil tetap kecil), sedangkan karakter non-huruf seperti spasi atau tanda baca dipertahankan tanpa perubahan sebelum string diakhiri dengan karakter `\0`.